Week-2 Cyclic Redundancy Check (CRC):

Cyclic Redundancy Check (CRC) is an error detection technique used in the **Data Link Layer** to detect errors that may occur during data transmission. CRC is based on **Modulo-2 (XOR) division**, where the sender treats the data as a binary number and divides it by a predefined **generator polynomial (divisor)**. The remainder obtained after the division is called the **CRC remainder** or **Frame Check Sequence (FCS)**.

In this program, the user enters the **binary dataword** and the **generator polynomial (divisor)**. The program first appends **(n−1) zeros** to the dataword, where **n** is the length of the divisor. These appended zeros provide space for the CRC bits that will be generated after the division.

The program then copies the modified data into a temporary array and performs **Modulo-2 (XOR) division** between the dividend and the divisor. Unlike ordinary binary division, CRC uses only the XOR operation, where:

- **0 XOR 0 = 0**
- **0 XOR 1 = 1**
- **1 XOR 0 = 1**
- **1 XOR 1 = 0**

After completing the XOR division, the program extracts the final **(n−1)** bits as the **CRC remainder**. These CRC bits are then appended to the original dataword to form the **transmitted codeword**.

At the receiver side, the received codeword is divided by the same generator polynomial. If the remainder obtained is **all zeros**, the data is considered to be error-free. Otherwise, the receiver concludes that an error has occurred during transmission.

Thus, this program demonstrates the implementation of the CRC algorithm for generating the CRC remainder and the transmitted codeword, which are used for reliable error detection in computer networks.

Algorithm

1. Start the program.
2. Read the binary dataword from the user.
3. Read the generator polynomial (divisor).
4. Calculate the lengths of the dataword and divisor.
5. Append **(divisor length − 1)** zeros to the dataword.
6. Copy the modified data into a temporary array.
7. Perform Modulo-2 (XOR) division between the temporary data and the divisor.
8. Extract the last **(divisor length − 1)** bits as the CRC remainder.

9. Append the CRC remainder to the original dataword to obtain the transmitted codeword.
10. Display the CRC remainder and the transmitted codeword.
11. Stop.

Expected Output

For example,

Input

Data : 1101011011

Divisor : 10011

Output

CRC Remainder : 1110

Transmitted Codeword : 11010110111110